

Agents

Generative AI

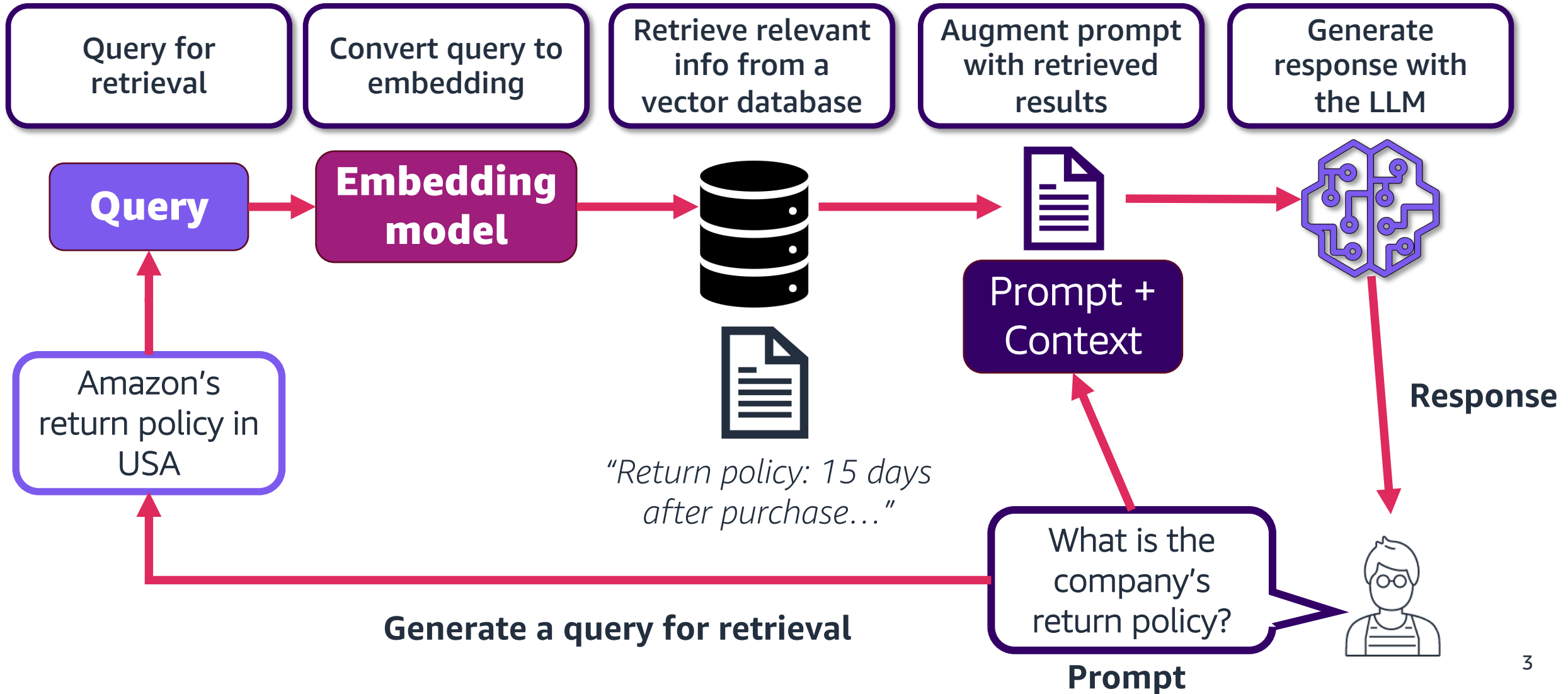
Module 3 – Lesson 4

Today's activities

- Review of lesson 3
- Tools and Agents
- LangChain Agents
- Agentic workflow



Review: RAG architecture



Tools and Agents

Key questions

- Can I use an LLM to do “**more**” than producing language?
- How to do **math** with an LLM? How to **query** a database with LLMs?
- Is it possible to **combine** the LLM with further systems and enhance the model’s capabilities?

Towards agents

⚙ So far:

- » **Prompting**: Single step inference
- » **Chains**: Pre-defined sequence of predictions

⚙ **Agents**

- » Automatically infer the sequence of actions to take
- » LLMs as reasoning engines

Agents

- AI systems that **automatically figure out how to best solve tasks** posed by the user
- Powered by **LLMs** as their reasoning engine whose capabilities are enhanced with **tools**
- Used to greatly **accelerate development and application** of generative AI applications

Tools

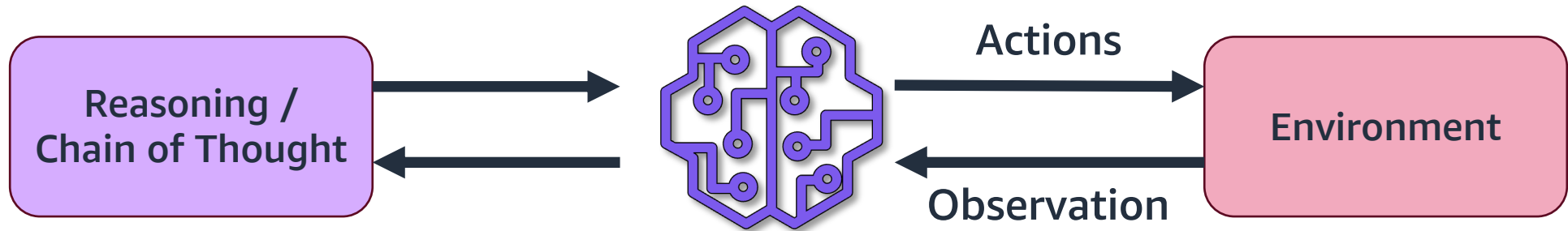
- Functions or interfaces that **the agent can interact with**
 - Can be APIs, document loaders, functions, other agents...
- Tools **may or may not** depend on an LLM
 - Example: Knowledge base containing company documents (RAG)
 - Example: Calculator / math tool to perform arithmetic operations
 - Example: API call to retrieve weather information
 - Example: Wikipedia tool to search for facts

What can agents do?

- **Understand** the request in natural language text using LLMs
- **Generate** a plan using prompting techniques such as chain-of-thought
- **Identify** required resources (like APIs, data sources, tools)
- **Execute** the plan (API and tools invocation)
- **Overcome** obstacles (Retry)

ReAct: Reason + Act

- LLMs often demonstrate the ability to **reason** logically.
- LLMs are capable of generating **action** plans.
- Combine both!
 - **Reasoning**: Create, track, and update action plans. Handle errors.
 - **Acting**: Interface with functions, tools, knowledge bases, or environments



ReAct = Reason + Act

Agentic tasks

- **Knowledge-intensive tasks**
 - Simple prompting suffers from fact hallucination
 - Agents can identify the data sources, write queries and retrieve relevant data
- **Decision-making tasks**
 - LLM planning capabilities are enhanced by custom tools
 - Selecting appropriate tools to execute tasks
 - However, it's still far from the performance of expert humans

LangChain Agents

LangChain tools

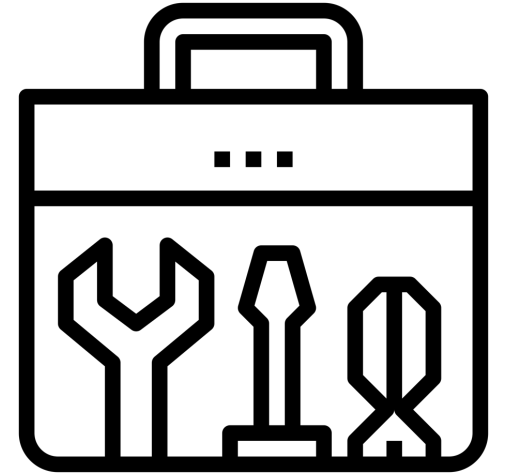
- **Functions or interfaces** the agent can interact with
 - Can be APIs, loaders, functions, other chains or agents
- Tools may or may not depend on an LLM
 - Example: Math tool uses an LLM for answering math questions
- LangChains supports third party integrations

```
# List of tool names
tool_names = [...]

# Load tools
tools = load_tools(tool_names, llm)
```

LangChain Toolkits

- Pre-defined set of tools
 - Accomplish specific objectives
- Some toolkits Integrations:
 - **GitHubToolkit**: Interact with Github repository
 - **JSON Toolkit**: Interacting with JSON or dict objects
 - **PowerBI Toolkit**: Interact with Power BI dataset
 - **PythonREPL** : Write and execute Python code
 - **SparkSQL Toolkit** : General inquiries about Spark SQL
 - **Jira Toolkit**: Interact with Jira instance



LangChain Agents

- Responsible for deciding the actions to take and then execute them
- Agents may have access to tools
 - Tools need to be described so that the agent is aware of its function
 - Important to give the agent access to right tools for a task
- **Agent runtimes** are responsible for calling the agent and executing the actions
 - A common one is the **AgentExecutor**

Limitations of agents

- Agents require **complex models** with reasoning capabilities
 - Often fails when using smaller or cheaper LLMs
- Higher level of **abstraction**
 - Challenging to rectify the intermediate steps
 - Difficult to debug
- **Sensitive to adversarial** examples and edge cases
 - Can pose security risks
- Agents can make simple tasks unnecessarily complex

Agentic workflow

Example: Q&A task

Query: “*What is the age of the current U.S. president today?*”

1. Find today's date
2. Find who is the U.S. president on that date (today)
3. Find that person's date of birth
4. Calculate the date difference between today's date and their date of birth
5. Convert answer in step 4 to a format specifying years, months, and days
6. Construct and send response to customer

How would a traditional LLM solve this?

Example: WITHOUT agents

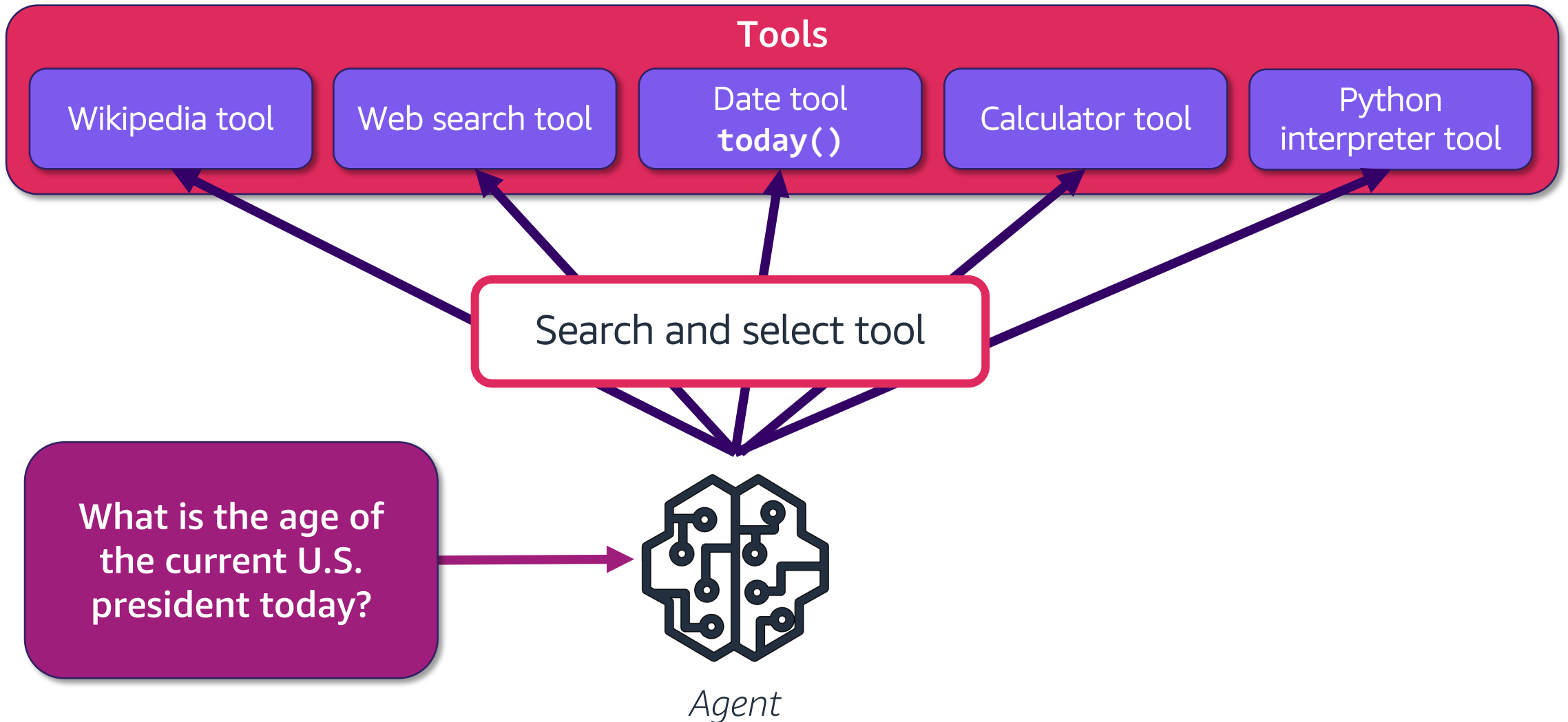
- Typical LLM answers to this question are inaccurate/not useful
- Try out the question with out-of-the-box LLMs like Mistral, Llama, Claude and others using Amazon Bedrock Chat:
- Example Mistral Large response (October 2024):

*As of today, the current U.S. President, Joe Biden, was born on November 20, 1942. To calculate his age, we would subtract his birth date from today's date and then convert that number into years. However, **since I'm an AI and don't have real-time capabilities, I can't provide the exact age right now.** You can use a date calculator or simply search for "Joe Biden's age" in a search engine to get the most accurate and up-to-date information.*

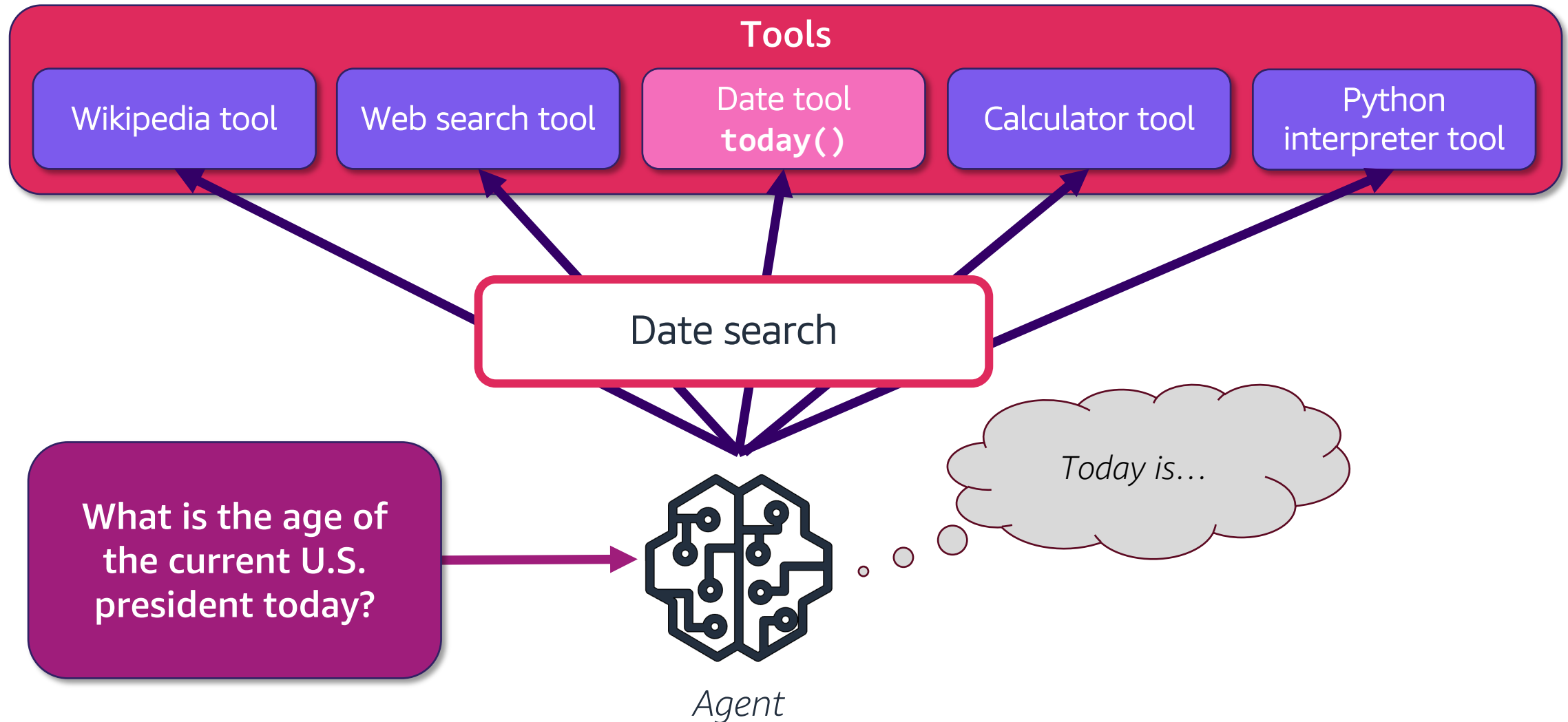
Example: WITH agents

- With Agents, using something like the ReAct technique:
 - A plan will be generated
 - Step-by-step execution of that plan will be performed
 - The model will give an accurate answer everyday with no additional code/training/deployment
 - Remember, today's date in the user question will change everyday

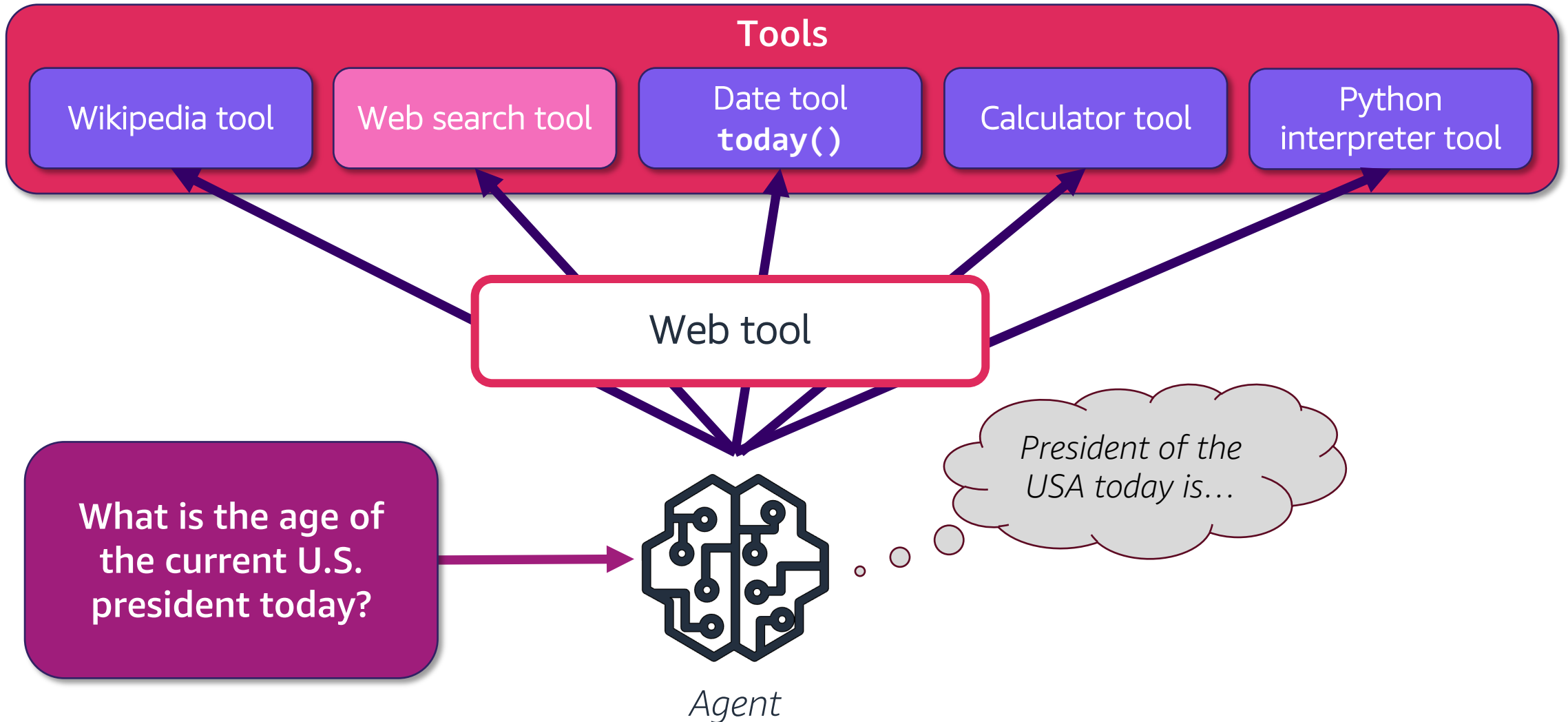
Agents with tools (1/6)



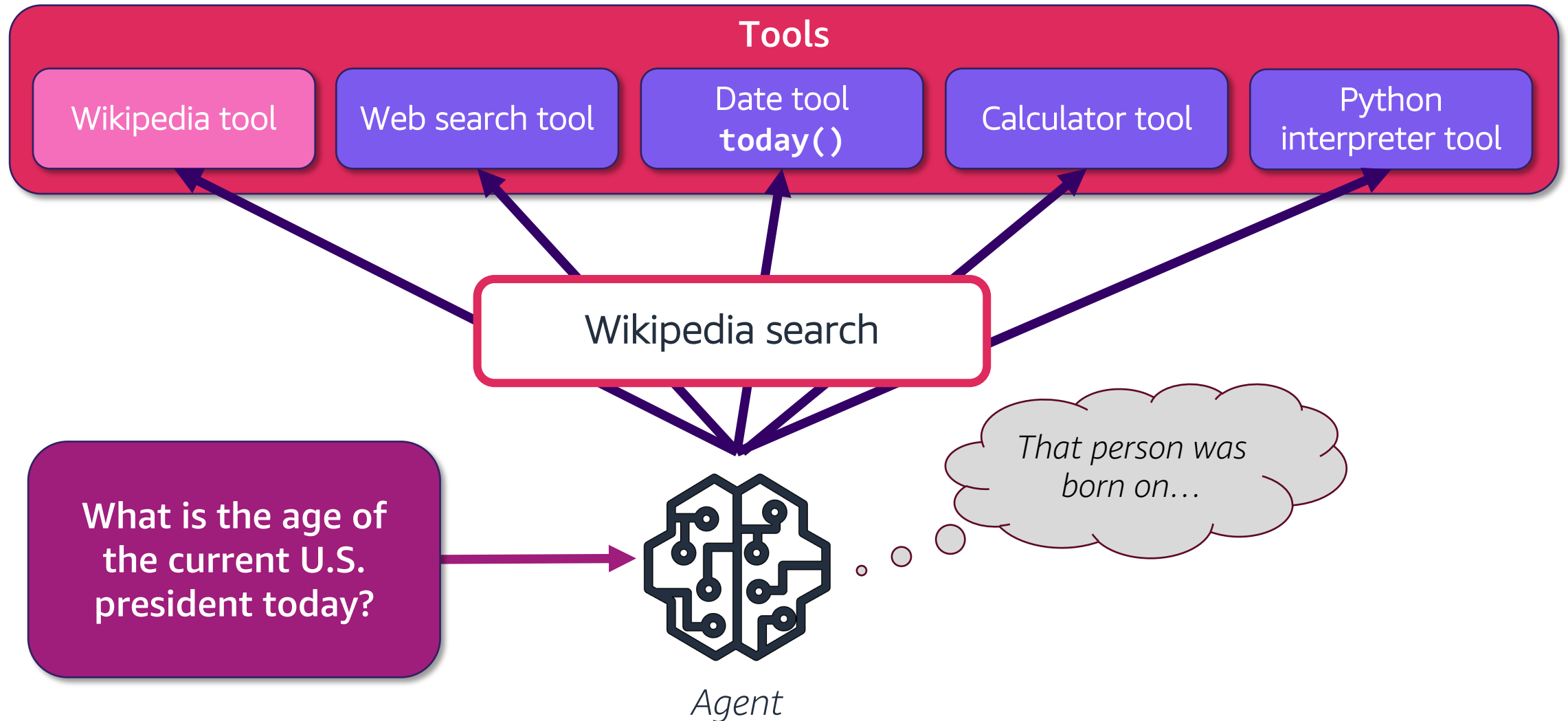
Agents with tools (2/6)



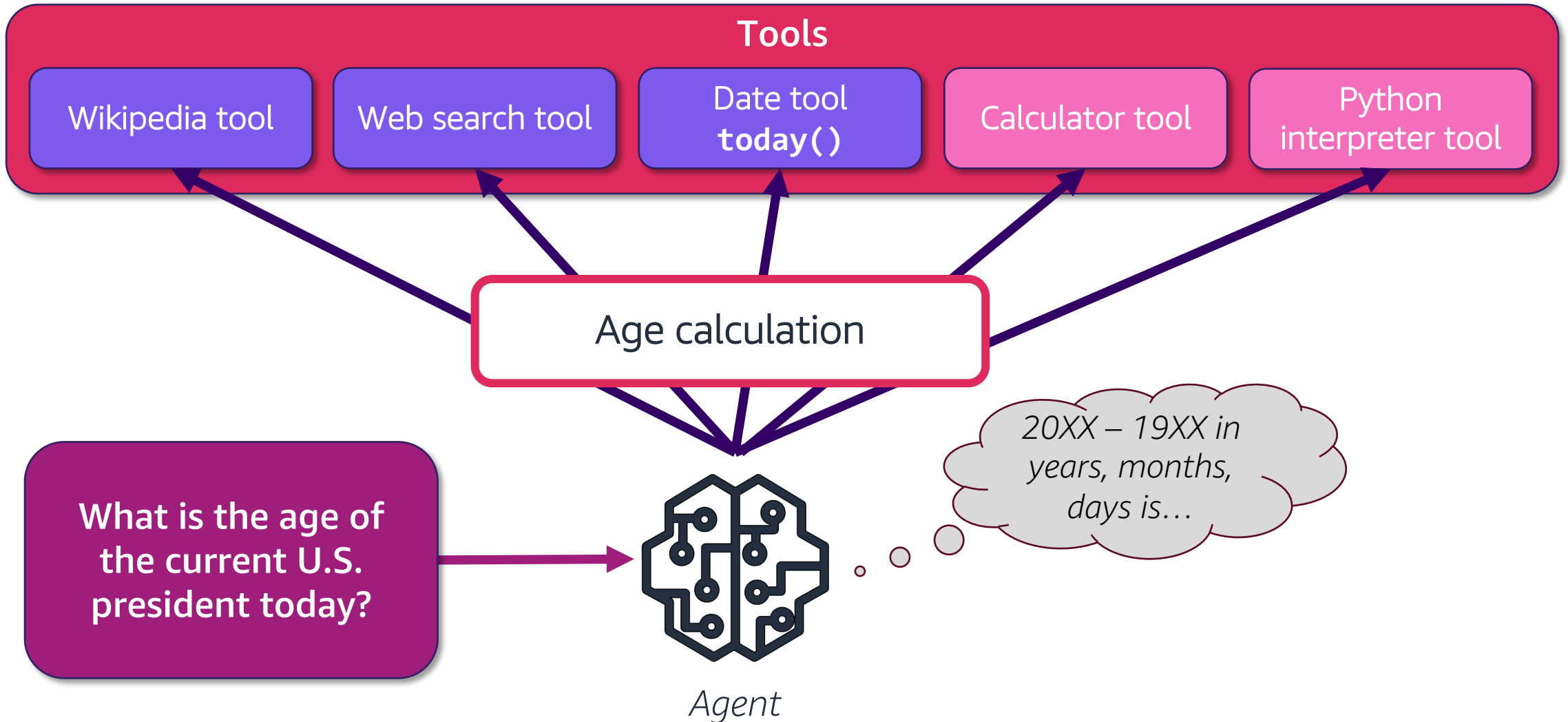
Agents with tools (3/6)



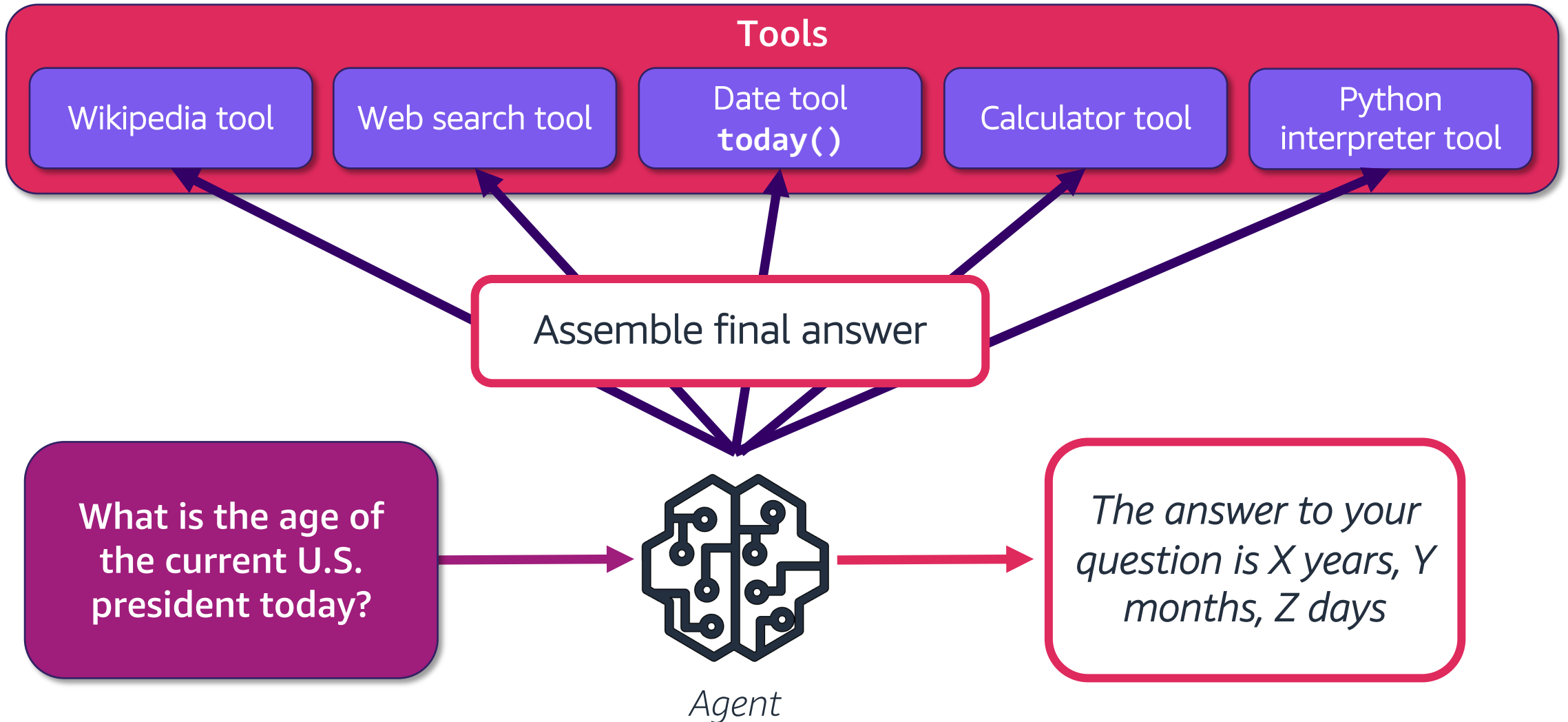
Agents with tools (4/6)



Agents with tools (5/6)

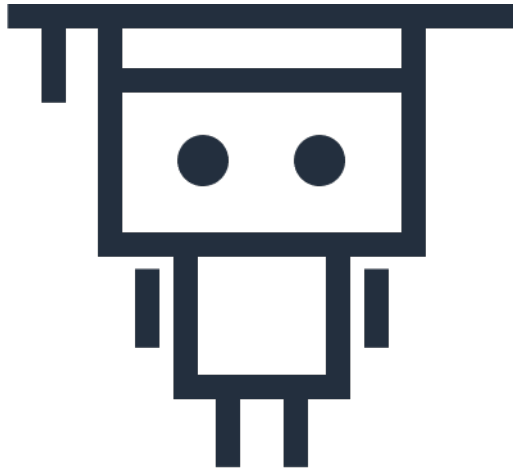


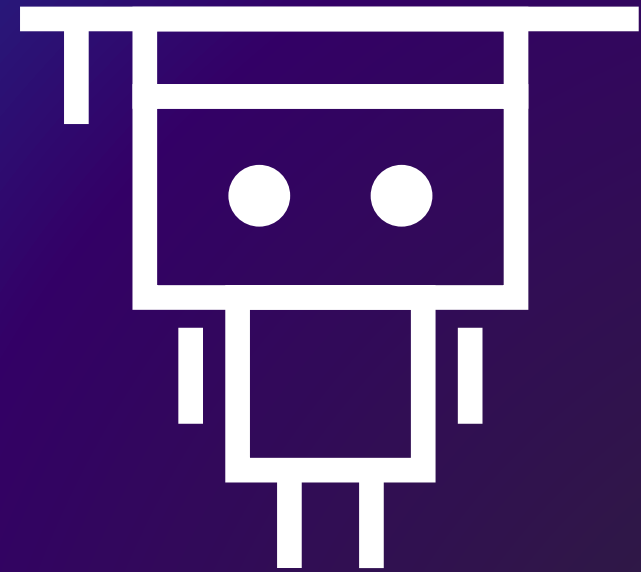
Agents with tools (6/6)



Next lesson

- This lesson covered topics related to agents.
- In the next lesson, you will learn about multimodal applications.





Thank you!